



Ensayo de Investigación:

Motor V8 de JavaScript y Optimización de la Distribución de Agua por Cisternas en Barrios de Loja

Matias Encarnación, Matías Lavanda, Julián Merino, Leonardo Sarango, Sherman Abarca

Carrera de Computación
Universidad Nacional de Loja

8 de octubre de 2026

Resumen

Este ensayo analiza dos componentes relacionados con una propuesta de solución informática para la distribución de agua mediante camiones cisterna en barrios periféricos de Loja. Primero, se estudia el funcionamiento del motor V8 de JavaScript, utilizado por Google Chrome y Node.js, con especial atención a la relación conceptual entre la Pila de Llamadas (Call Stack) y el Montón de Memoria (Memory Heap). Se explica por qué es útil asociar la pila con los contextos de ejecución y el montón con los objetos dinámicos, pero también se aclara que esta separación no constituye una regla física absoluta del motor. Después se presenta el modelo de la red de distribución de agua como un grafo dirigido con capacidades y se analiza el uso de Edmonds–Karp, red residual y corte mínimo para identificar la cantidad máxima de agua distribuible y los cuellos de botella. Finalmente, se relacionan las decisiones de estructura de datos y memoria con el rendimiento esperado de un sistema desarrollado con Node.js, Express, React y Mapbox GL.

1. Introducción

La evolución de JavaScript ha convertido al lenguaje en una tecnología utilizada tanto en el navegador como en servidores. V8 es el motor de JavaScript y WebAssembly de código abierto desarrollado por Google; se emplea, entre otros entornos, en Chrome y Node.js [1]. Esta característica permite utilizar un mismo ecosistema tecnológico para construir interfaces web y servicios de backend.

En el proyecto de optimización de distribución de agua por cisternas en barrios de Loja, el documento base propone representar la red logística como un grafo de capacidades y utilizar Node.js, Express, React y Mapbox GL para implementar y visualizar la solución. El objetivo general consiste en construir un sistema de logística que permita optimizar la

entrega de agua potable desde plantas de tratamiento hacia barrios afectados por cortes del servicio. Estos elementos corresponden al documento base proporcionado para la investigación.

Desde la perspectiva de la computación, el problema no se limita a implementar un algoritmo de caminos o flujo máximo. También es necesario comprender cómo el entorno de ejecución administra memoria y cómo las estructuras de datos elegidas afectan el rendimiento. Por ello, el estudio de Call Stack, Memory Heap, recolección de basura y optimización de V8 permite relacionar la teoría del motor de ejecución con las necesidades concretas del sistema logístico.

2. Funcionamiento del motor V8 de JavaScript

V8 implementa ECMAScript y WebAssembly, ejecuta código JavaScript, administra la asignación de memoria de los objetos y realiza recolección de basura cuando los objetos dejan de ser necesarios [1]. Es importante distinguir el motor del entorno que lo integra: Chrome proporciona APIs del navegador, como el DOM, mientras que Node.js proporciona APIs orientadas al servidor y un modelo de ejecución basado en el event loop [2].

2.1. Proceso general de ejecución

De forma simplificada, el código JavaScript pasa por una cadena de procesamiento que incluye análisis del código y diferentes niveles de ejecución y compilación. V8 utiliza Ignition como intérprete de bytecode y dispone de compiladores JIT para optimizar código que presenta comportamiento suficientemente estable o frecuente. La documentación técnica de V8 describe la evolución de la arquitectura desde la combinación Ignition-TurboFan y posteriormente la incorporación de Sparkplug y Maglev [3], [4], [5].

El proceso puede resumirse así:

1. El código fuente JavaScript es analizado y convertido a una representación que puede ser procesada por el motor.
2. Ignition ejecuta bytecode y recopila información sobre el comportamiento del programa.
3. Los compiladores de niveles superiores pueden generar código máquina optimizado para funciones que resultan calientes (*hot*).
4. Si las suposiciones empleadas durante una optimización dejan de cumplirse, V8 puede desoptimizar y volver a un nivel de ejecución apropiado.

Por tanto, no es técnicamente exacto presentar a V8 únicamente como “parser → Ignition → TurboFan”. Ese esquema es útil para introducir el funcionamiento, pero las ver-

siones modernas de V8 incorporan más niveles de ejecución, como Sparkplug y Maglev [4], [5].

3. Pila de llamadas y montón de memoria

3.1. Call Stack

La Pila de Llamadas es una estructura de ejecución de tipo LIFO (*Last In, First Out*). Cada llamada a una función genera un contexto o marco de ejecución (*stack frame*) y, cuando la función termina, ese marco deja de ser el contexto activo. MDN describe la pila como la estructura que permite entrar y salir de contextos de ejecución [6].

Por ejemplo:

```
function sumar(a, b) {  
  return a + b;  
}  
  
function calcular() {  
  const x = 10;  
  const y = 20;  
  return sumar(x, y);  
}  
  
console.log(calcular());
```

Código 1: Ejemplo de llamadas anidadas y Call Stack

Cuando se ejecuta `calcular()`, su marco entra en la pila. Después, al invocarse `sumar()`, se crea otro marco encima. Al terminar `sumar()`, su marco se retira y continúa `calcular()`.

3.2. Memory Heap

El Heap es una región de memoria dinámica donde se alojan objetos creados durante la ejecución. MDN lo describe como una región grande y mayormente no estructurada que se puebla a medida que se crean objetos [6]. En este modelo se ubican estructuras como objetos, arreglos y funciones, además de datos y metadatos internos gestionados por el motor.

El siguiente ejemplo crea estructuras que requieren almacenamiento dinámico:

```
const cisterna = {  
  placa: "ABC-1234",  
  capacidad: 5000,  
  barrio: "Motupe"  
};  
  
const rutas = [  
  { origen: "Pucara", destino: "Motupe", capacidad: 5000 },
```

```
{ origen: "Carigan", destino: "Zalapa", capacidad: 3000 }  
];
```

Código 2: Ejemplo de objetos y arreglos

3.3. Clases ocultas e Inline Caching

La administración de objetos en V8 no se limita a reservar espacio en el heap. El motor también mantiene información sobre la forma de los objetos mediante estructuras internas conocidas como *HiddenClasses* o *Maps*. Objetos que presentan propiedades con estructuras compatibles pueden compartir una clase oculta, lo que facilita el acceso eficiente a sus propiedades [7].

Por ejemplo, en un sistema logístico conviene que las instancias de una misma entidad se construyan de forma consistente:

```
class Cisterna {  
  constructor(placa, capacidad, origen) {  
    this.placa = placa;  
    this.capacidad = capacidad;  
    this.origen = origen;  
  }  
}  
  
const c1 = new Cisterna("ABC-1234", 5000, "Pucara");  
const c2 = new Cisterna("XYZ-5678", 3000, "Carigan");
```

Código 3: Construcción consistente de objetos logísticos

Cuando los objetos mantienen formas compatibles, V8 puede aprovechar mecanismos como los *inline caches* para acelerar accesos repetidos a propiedades. Esta optimización conecta directamente el diseño de las estructuras de datos con el rendimiento del algoritmo, aunque el resultado concreto depende de la versión de V8 y del comportamiento real de la aplicación [7].

3.4. Relación entre Stack y Heap

El modelo didáctico más utilizado representa las variables primitivas y los contextos de ejecución en la pila, mientras que los objetos y arreglos se representan en el montón. Una referencia asociada a una variable permite llegar al objeto correspondiente. Esta explicación es útil para comprender el concepto, pero no debe interpretarse como una garantía de que cada valor primitivo se almacena físicamente en el stack y cada cadena necesariamente en el heap.

V8 utiliza representaciones internas optimizadas. Por ejemplo, determinados valores pequeños pueden representarse de forma inmediata y las cadenas pueden tener representaciones internas diferentes. MDN también advierte que la gestión de memoria es automática y que el motor puede tomar decisiones de representación que no son visibles

para el programador [8]. Por ello, la distinción Stack/Heap debe utilizarse como un modelo conceptual de ejecución y no como un mapa físico exacto de cada byte de memoria.

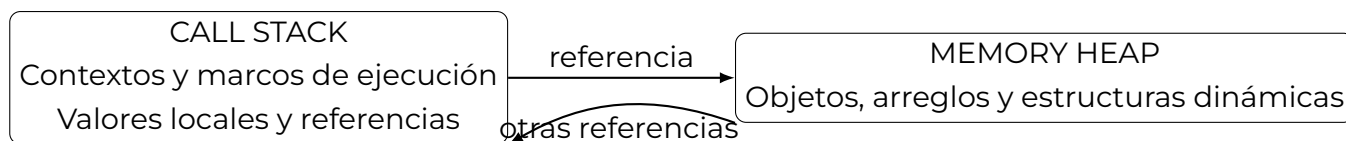


Figura 1: Modelo conceptual de la relación entre Call Stack y Memory Heap. Elaboración propia con base en [6], [8].

3.5. Primitivos, objetos y paso de argumentos

JavaScript distingue valores primitivos de objetos. Entre los primitivos se encuentran `number`, `string`, `boolean`, `undefined`, `null`, `bigint` y `symbol`; los objetos incluyen estructuras como `Array` y funciones [9].

Una diferencia observable es la forma en que se comporta la asignación:

```
let capacidad = 5000;
let copia = capacidad;
copia = 3000;

console.log(capacidad); // 5000

const cisternaA = { capacidad: 5000 };
const cisternaB = cisternaA;
cisternaB.capacidad = 3000;

console.log(cisternaA.capacidad); // 3000
```

Código 4: Diferencia observable entre valores primitivos y objetos

En el primer caso, cambiar `copia` no modifica `capacidad`. En el segundo, ambas variables permiten acceder al mismo objeto. Conviene hablar de “paso por valor de una referencia” en el caso de objetos, y no de un verdadero “paso por referencia”, porque MDN establece que los argumentos de funciones se pasan por valor [10].

4. Gestión automática de memoria y problemas asociados

V8 utiliza un recolector de basura generacional. Su hipótesis central es que muchos objetos tienen una vida corta, por lo que el heap se organiza conceptualmente en una generación joven y otra antigua. Las colecciones frecuentes de la generación joven permiten recuperar objetos de corta duración sin recorrer constantemente toda la generación antigua [11].

En un sistema como el propuesto para Loja, pueden crearse temporalmente objetos durante el cálculo de caminos, actualización de capacidades residuales, construcción de

respuestas JSON y procesamiento de solicitudes. Una implementación que genere grandes cantidades de objetos temporales puede aumentar la presión sobre el recolector de basura.

El problema inverso aparece cuando el programa mantiene referencias innecesarias. Una fuga de memoria (*memory leak*) puede producirse cuando objetos que ya no deberían utilizarse continúan siendo alcanzables desde otras referencias. MDN explica que la recolección automática depende de determinar qué memoria continúa siendo necesaria [8].

En el caso de la pila, una recursión excesiva puede producir un error como `RangeError: Maximum call stack size exceeded`. En cambio, el agotamiento del heap está asociado con la acumulación de objetos y estructuras dinámicas que incrementan el consumo de memoria.

5. Implicaciones de memoria para el proyecto de agua de Loja

El documento base establece que el grafo debe implementarse mediante listas de adyacencia en lugar de matrices para reducir el consumo de memoria en redes dispersas. Esta decisión es coherente con la necesidad de evitar estructuras innecesariamente grandes cuando la cantidad de conexiones es mucho menor que $|V|^2$.

Además, se propone que el sistema pueda registrar plantas, puntos de recarga, barrios, rutas y cisternas; ejecutar Edmonds–Karp; detectar rutas saturadas mediante Min-Cut; visualizar el resultado sobre un mapa y recalcular la solución ante cambios de capacidad o disponibilidad.

La relación con V8 aparece en tres niveles:

- **Representación:** los nodos y aristas pueden modelarse mediante objetos y arreglos.
- **Cómputo:** la búsqueda en anchura de Edmonds–Karp genera recorridos y estructuras temporales que requieren memoria.
- **Concurrencia:** Node.js utiliza un event loop para procesar operaciones de E/S sin bloquear el hilo de JavaScript por cada solicitud [2].

Sin embargo, una advertencia es fundamental: Edmonds–Karp es un algoritmo intensivo en CPU. Si el cálculo ocupa demasiado tiempo en el hilo principal, puede afectar la capacidad de respuesta del panel web. Por eso, para redes grandes o simulaciones concurrentes, deben evaluarse estrategias como particionamiento, procesos o workers, dependiendo de los requisitos reales.

6. Optimización de la distribución de agua por cisternas en barrios de Loja

6.1. Problema y objetivos

El problema se formula como la necesidad de distribuir agua potable mediante camiones cisterna hacia barrios periféricos afectados por cortes de servicio. El objetivo es maximizar el volumen de agua entregable respetando capacidades de rutas, fuentes y vehículos.

El documento base establece como objetivos específicos modelar la red sectorizada como un grafo de capacidades, implementar Edmonds–Karp y desarrollar un panel web con Express y Mapbox para visualizar la asignación volumétrica y las rutas saturadas.

Entre los requerimientos funcionales se encuentran el registro de nodos y rutas, alta de cisternas, estimación de demanda, cálculo del flujo máximo, identificación del corte mínimo, visualización geográfica y alerta cuando la demanda supera el flujo máximo calculado.

6.2. Modelo matemático

La red puede representarse como un grafo dirigido de capacidades:

$$G = (V, E),$$

donde V representa plantas, puntos de recarga y barrios, mientras que E representa rutas de transporte. A cada arista (u, v) se le asigna una capacidad $c(u, v)$.

El flujo $f(u, v)$ debe cumplir:

$$0 \leq f(u, v) \leq c(u, v), \quad \forall (u, v) \in E.$$

Para un nodo intermedio se debe conservar el flujo:

$$\sum_w f(w, u) = \sum_v f(u, v), \quad u \in V \setminus \{s, t\}.$$

Esta formulación permite interpretar la planta o conjunto de plantas como fuente s y los barrios necesitados como sumidero t .

6.3. Edmonds–Karp y red residual

Edmonds–Karp es una implementación de Ford–Fulkerson que selecciona caminos aumentantes mediante Búsqueda en Anchura (BFS). Para una arista, la capacidad residual puede expresarse como:

$$c_f(u, v) = c(u, v) - f(u, v).$$

Mientras exista un camino desde la fuente hasta el sumidero en la red residual, se determina la capacidad mínima del camino y se incrementa el flujo en esa cantidad.

```
while (existeCaminoAumentante(redResidual, fuente, sumidero)) {
    const camino = bfs(redResidual, fuente, sumidero);
    const delta = capacidadMinima(camino);
    aumentarFlujo(camino, delta);
    actualizarRedResidual(camino, delta);
}
```

Código 5: Esquema conceptual de Edmonds–Karp

El algoritmo permite obtener el volumen máximo que puede atravesar la red bajo las capacidades definidas. En el contexto de Loja, el resultado puede expresarse en litros o metros cúbicos por unidad de tiempo, siempre que todas las capacidades utilicen una unidad consistente.

6.4. Corte mínimo como identificación de cuellos de botella

El teorema de flujo máximo–corte mínimo establece que el valor del flujo máximo es igual a la capacidad del corte mínimo que separa la fuente del sumidero [12]. En el proyecto, las aristas del corte mínimo pueden interpretarse como cuellos de botella de la red.

Por ejemplo, si la capacidad de producción de una planta, una ruta vial o la disponibilidad de cisternas limita el flujo total, el corte mínimo puede ayudar a localizar dónde una ampliación de capacidad produciría un beneficio mayor. Esta interpretación convierte el algoritmo en una herramienta de apoyo a la toma de decisiones.

Debe distinguirse, sin embargo, entre el modelo matemático y la realidad operativa. Una red real de distribución también requiere restricciones de tiempo, tráfico, disponibilidad de conductores, capacidad de los vehículos, ventanas de atención y prioridades de los barrios. Por ello, el flujo máximo constituye una primera capa de optimización y puede complementarse posteriormente con problemas de ruteo de vehículos y restricciones temporales.

7. Requerimientos y restricciones del sistema

De acuerdo con el documento base, el sistema debe permitir registrar plantas, puntos de recarga, barrios y rutas con capacidades; registrar camiones cisterna con placa y capacidad; asignar demandas volumétricas; ejecutar Edmonds–Karp; obtener el flujo máximo y el flujo por arista; identificar rutas saturadas mediante Min-Cut; visualizar la red sobre un mapa real de Loja; modificar capacidades y recalcular; y generar una alerta cuando la demanda total supere el flujo máximo.

Entre los requerimientos no funcionales definidos en el documento se encuentran un tiempo objetivo inferior a 1.5 segundos para el cálculo con hasta 500 nodos, el uso de listas de adyacencia para redes dispersas, una interfaz de mapa con al menos 30 FPS, una arquitectura modular React–Express mediante API REST JSON, un backend sin estado para simulaciones concurrentes, compatibilidad con navegadores modernos y validación estructural del grafo mediante herramientas como Zod o Joi.

Estas condiciones deben interpretarse como especificaciones del proyecto. No constituyen resultados experimentales: el cumplimiento del tiempo de 1.5 segundos o de 30 FPS debe verificarse mediante pruebas controladas con datos y hardware definidos.

8. Arquitectura tecnológica propuesta

La arquitectura planteada en el documento base utiliza React para la interfaz, Express.js y Node.js para el backend y Mapbox GL para la visualización geográfica. La comunicación puede realizarse mediante una API REST JSON.

1. **Frontend:** React presenta barrios, plantas, rutas, cisternas, demandas y resultados.
2. **Backend:** Node.js y Express reciben solicitudes, validan datos y ejecutan los algoritmos.
3. **Motor algorítmico:** Edmonds–Karp calcula el flujo máximo y una etapa posterior identifica el corte mínimo.
4. **Visualización:** Mapbox GL representa fuentes, barrios y rutas, diferenciando las aristas saturadas.
5. **Validación:** bibliotecas como Zod o Joi pueden validar capacidades positivas, nodos válidos y estructuras del grafo antes del cálculo.

Water Distribution Optimization System

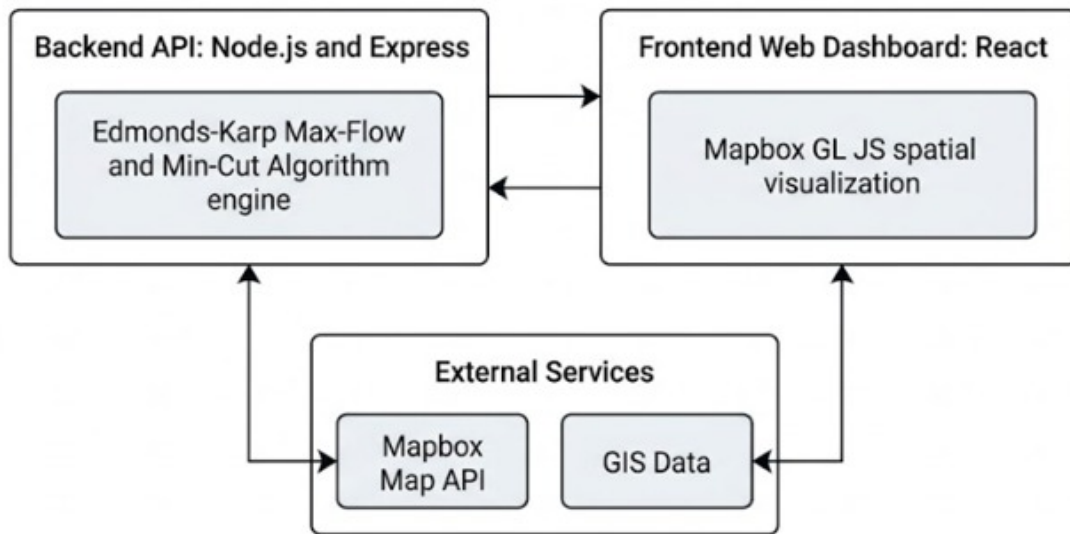


Figura 2: Arquitectura tecnológica propuesta para el sistema de optimización de distribución de agua.

El documento base fija además metas de rendimiento para el prototipo, como calcular el flujo en menos de 1.5 segundos para hasta 500 nodos y mantener fluidez visual en el mapa. Estas cifras deben considerarse objetivos de ingeniería del proyecto, no resultados experimentales ya demostrados.

9. Análisis comparativo de Stack y Heap

Tabla 1: Comparación conceptual entre Call Stack y Memory Heap

Característica	Call Stack	Memory Heap
Función principal	Mantener contextos y marcos de ejecución de llamadas	Mantener almacenamiento dinámico de objetos y estructuras
Organización	LIFO	Región dinámica, no equivalente a una pila
Ejemplos didácticos	Variables locales, parámetros y referencias	Objetos, arreglos, funciones y estructuras dinámicas
Ciclo de vida	Asociado a la entrada y salida de contextos	Depende de la alcanzabilidad y del recolector de basura
Problema típico	Recursión profunda y desbordamiento de pila	Retención excesiva de objetos y agotamiento de memoria
Precisión de la representación	No debe asumirse que todo valor está físicamente aquí	No debe asumirse que todos los datos de JavaScript están aquí de una única manera

La Tabla 1 presenta una comparación conceptual. La precisión es importante: Stack y Heap son abstracciones útiles para explicar el modelo de ejecución, pero las representaciones internas de V8 son más complejas y pueden cambiar con las optimizaciones del motor.

10. Discusión

La relación entre V8 y el sistema de distribución de agua muestra que la eficiencia de un algoritmo no depende únicamente de su complejidad matemática. También depende de cómo se representan los datos, cuántos objetos temporales se crean, cuánto tiempo permanece ocupado el hilo principal y cómo se gestiona la memoria.

Para la red de Loja, una estructura basada en listas de adyacencia resulta apropiada para grafos dispersos. Asimismo, mantener objetos con estructuras previsibles puede facilitar las optimizaciones internas de V8. No obstante, las afirmaciones cuantitativas del documento base sobre reducción de tiempo, instrucciones o memoria mediante desacoplamiento de tipos deben considerarse resultados experimentales del material proporcionado y no generalizarse a cualquier versión de V8 sin reproducir el experimento.

Otro aspecto relevante es Node.js. Su event loop facilita el tratamiento de muchas operaciones de entrada/salida con un solo hilo de JavaScript, pero no elimina el costo de operaciones intensivas en CPU [2]. Por ello, si Edmonds–Karp se ejecuta sobre una red suficientemente grande y bloquea el hilo principal, la interfaz y las demás solicitudes pueden experimentar latencia.

11. Resultados cuantitativos consignados en el documento base

El documento base incluye una tabla de datos empíricos atribuida a pruebas JSBench sobre desacoplamiento de tipos en V8. Allí se reportan una reducción del 36 % en el tiempo de ejecución, una reducción del 49 % en instrucciones dinámicas, un ahorro del 19.9 % en memoria heap asignada, un incremento de 18 puntos porcentuales en la tasa global de aciertos de Inline Cache (79 % a 97 %) y un incremento de 14 puntos porcentuales en la tasa de aciertos monomórficos (72 % a 86 %). También se reporta una sobrecarga de 4.3 % de tiempo de Garbage Collection atribuida al rastreo de punteros.

Estos valores se conservan porque forman parte del material proporcionado; sin embargo, no se presentan aquí como una ley general de V8. Para convertirlos en evidencia reproducible sería necesario disponer del código de prueba, versión exacta de V8, hardware, parámetros del benchmark y procedimiento estadístico.

12. Conclusiones

El estudio permite establecer que V8 es una pieza fundamental para comprender la ejecución de JavaScript en Chrome y Node.js. La Pila de Llamadas organiza los contextos de ejecución siguiendo el principio LIFO, mientras que el Heap representa conceptualmente la zona de almacenamiento dinámico en la que se gestionan objetos y otras estructuras. Esta separación es especialmente útil para comprender el ciclo de vida de las variables y objetos, aunque no debe interpretarse como una regla física rígida sobre la ubicación de cada valor.

La gestión automática de memoria de V8, junto con su recolector generacional, permite que JavaScript administre objetos sin que el programador tenga que liberar manualmente cada asignación. Sin embargo, una aplicación todavía puede sufrir desbordamientos de pila, consumo excesivo del heap y fugas de memoria si conserva referencias innecesarias o produce estructuras temporales de manera descontrolada.

Para la distribución de agua por cisternas en Loja, la formulación como un grafo de capacidades proporciona una base matemática para estudiar la cantidad máxima de agua distribuible. Edmonds–Karp permite calcular el flujo máximo mediante la red residual, mientras que el corte mínimo ayuda a identificar los cuellos de botella que limitan la capacidad de distribución.

Finalmente, la combinación de Node.js, Express, React y Mapbox GL ofrece una arquitectura coherente para un prototipo de sistema logístico. No obstante, para convertir el modelo en una solución operativa sería necesario incorporar datos viales y de demanda reales, restricciones de horarios y vehículos, validación con escenarios de prueba y mediciones reproducibles de rendimiento y memoria.

Nota sobre las fuentes y el alcance

El contenido específico del proyecto de Loja se construyó a partir del documento de información consultada proporcionado para esta tarea. Las afirmaciones técnicas sobre V8, JavaScript, Node.js, gestión de memoria y flujo máximo se contrastaron con documentación técnica y bibliografía externa. Los ejemplos de barrios, plantas y capacidades se entienden como elementos del modelo propuesto y no como mediciones oficiales de la red municipal.

13. Declaración de IA

Dentro de la realización del proyecto se hizo uso del asistente de IA de Overleaf, el cual usa un modelo de IA de GPT-3 para realizar la división de los distintos temas a abarcar por persona y realizar un trabajo en grupo organizado. Adicionalmente, se hizo uso de Gemini 3.8 Flash para la generación del diagrama de la arquitectura en base a la información consultada.

Referencias

- [1] V8 Team. «V8 Documentation,» Google, visitado 7 de oct. de 2026. dirección: <https://v8.dev/docs>.
- [2] Node.js Contributors. «The Node.js Event Loop,» Node.js, visitado 7 de oct. de 2026. dirección: <https://nodejs.org/learn/asynchronous-work/event-loop-timers-and-nexttick>.
- [3] M. Lippautz, P. Pizlo et al. «Launching Ignition and TurboFan,» V8, visitado 7 de oct. de 2026. dirección: <https://v8.dev/blog/launching-ignition-and-turbofan>.
- [4] V8 Team. «Sparkplug—a non-optimizing JavaScript compiler,» V8, visitado 7 de oct. de 2026. dirección: <https://v8.dev/blog/sparkplug>.
- [5] V8 Team. «Maglev—V8’s fastest optimizing JIT,» V8, visitado 7 de oct. de 2026. dirección: <https://v8.dev/blog/maglev>.
- [6] MDN Web Docs Contributors. «JavaScript execution model,» MDN Web Docs, visitado 7 de oct. de 2026. dirección: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model.
- [7] V8 Team. «Fast properties in V8,» V8, visitado 7 de oct. de 2026. dirección: <https://v8.dev/blog/fast-properties>.
- [8] MDN Web Docs Contributors. «Memory management,» MDN Web Docs, visitado 7 de oct. de 2026. dirección: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Memory_management.

- [9] MDN Web Docs Contributors. «JavaScript data types and data structures,» MDN Web Docs, visitado 7 de oct. de 2026. dirección: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures.
- [10] MDN Web Docs Contributors. «Functions,» MDN Web Docs, visitado 7 de oct. de 2026. dirección: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions>.
- [11] V8 Team. «Getting garbage collection for free,» V8, visitado 7 de oct. de 2026. dirección: <https://v8.dev/blog/free-garbage-collection>.
- [12] R. K. Ahuja, T. L. Magnanti y J. B. Orlin, «Network flows: Theory, algorithms, and applications,» *Prentice Hall*, 1993.